

FreightX

Autonomous SME Trade Finance & Cargo Passport on Arc HACKATHON SUBMISSION

Track: SME Trade Finance & Working Capital Workflow

Contents

1	Executive Summary & Problem Statement	2
2	Comprehensive Feature Walkthrough	3
3	Core System Architecture & Protocol Workflows	4
3.1	System Architecture Diagram	4
3.2	Detailed Protocol Workflow	5
4	Detailed Technical Stack & Smart Contracts	7
5	Circle Developer Products Integration Details	8
6	Exhaustive Circle Product Feedback & Recommendations	9
7	Setup, Deployment & Verification Guide	10

1 Executive Summary & Problem Statement

International trade and maritime shipping form the backbone of the global economy, yet Small and Medium Enterprises (SMEs) face massive barriers in this space. Three critical inefficiencies plague the global supply chain today:

A. The Trade Finance Gap

SMEs struggle to access pre-shipment Purchase Order (PO) financing and invoice factoring. Commercial banks require extensive paperwork, and credit scoring takes weeks, leading to missed trade opportunities.

B. Idle Transit Capital (Float Inefficiency)

Goods spend weeks at sea. During this time, buyers' locked escrow collateral remains completely dormant. Millions of dollars in capital sit idle, earning zero yield for the participants.

C. Operational Friction & Manual Checks

Verifying milestones (e.g., arrival at intermediate ports, customs clearance, and cold-chain temperature limits) relies on manual, error-prone reports. Disputes over spoiled cargo or cargo delays take months to settle, locking up valuable working capital.

FreightX is a next-generation decentralized logistics and trade finance protocol built on Circle's Arc Network. By leveraging programmable USDC, embedded wallets, yield-bearing USYC treasury sweeping, autonomous AI agents, and real-time IoT telemetry, FreightX automates the entire supply chain trade lifecycle—from purchase order creation to final port settlement.

2 Comprehensive Feature Walkthrough

FreightX combines trade logistics and decentralized finance into an integrated system with the following eight core capabilities:

1. Pre-Shipment PO Financing:

SMEs can request purchase order loans from capital providers. Investors fund loans instantly in USDC. When the buyer later establishes the trade escrow, the system automatically links to the active PO loan, locking the repayment mechanism directly into the escrow settlement waterfall.

2. Milestone-Gated Cargo Escrow:

Escrows are governed by smart contracts on the Arc Network. Instead of a binary release (all-or-nothing), FreightX supports automated progressive payouts. For instance, arriving at a major transshipment hub like Singapore automatically unlocks 30% of the collateral to the supplier to support their liquidity during transit.

3. Cargo Twin & Cryptographic Passports (ERC-721 + W3C VC):

Every registered cargo container is minted on-chain as a digital twin via the FreightPassport NFT contract. Metadata such as container numbers, weight, shipper details, and bill of lading (BoL) IPFS hashes are embedded. The app can export cryptographic W3C Verifiable Credentials (VCs) in JSON-LD format, containing cryptographic signatures to prove trade history.

4. Receivables Factoring Marketplace:

If a supplier faces cash flow constraints mid-transit, they can list their unpaid cargo invoice for sale at a discount. Investors can purchase these factored receivables in USDC, which instantly redirects the on-chain beneficiary address of the escrow payout to the investor's wallet.

5. Automated IoT Telematics & Demurrage:

Container sensors stream GPS coordinates and temperature logs. The system simulates time acceleration (e.g., 4 hours/sec) to monitor port customs clearance. If cargo exceeds its allocated "free time hours" at port, a real-time demurrage accumulator automatically calculates storage penalties and charges them against the escrow settlement.

6. Cold-Chain Thermal Breach Protection:

For temperature-sensitive cargo (like vaccines or fresh produce), container sensors report temperature logs. If the temperature breaches thresholds, the smart logic flags a "Thermal Breach" and automatically applies a pre-agreed penalty (e.g., 5% deduction) to the supplier's final payout.

7. USYC Idle Treasury Sweep:

To solve the problem of idle capital in transit, FreightX allows buyers to opt-in to sweep their locked escrow USDC collateral into a simulated USYC (yield-bearing tokenized treasury) vault. While the ship travels, the vault accrues yield. Upon final delivery, the principal is settled to the supplier, and the accrued yield is refunded back to the buyer.

8. AI Agent Autopilot & Nanopayments (ERC-8004 & ERC-8183):

FreightX introduces an "AI Agent Logistics Coordinator" that acts on behalf of the trade stakeholders. The agent queries telemetry data through an API protected by an HTTP 402 Paywall (Nanopayments). The agent pays sub-cent USDC micropayments to read sensor logs, evaluates cold-chain violations, and submits cryptographic proofs to trigger automated milestones on-chain.

3 Core System Architecture & Protocol Workflows

3.1 System Architecture Diagram

Below is the structured layout and data flow schematic of the FreightX supply chain platform:

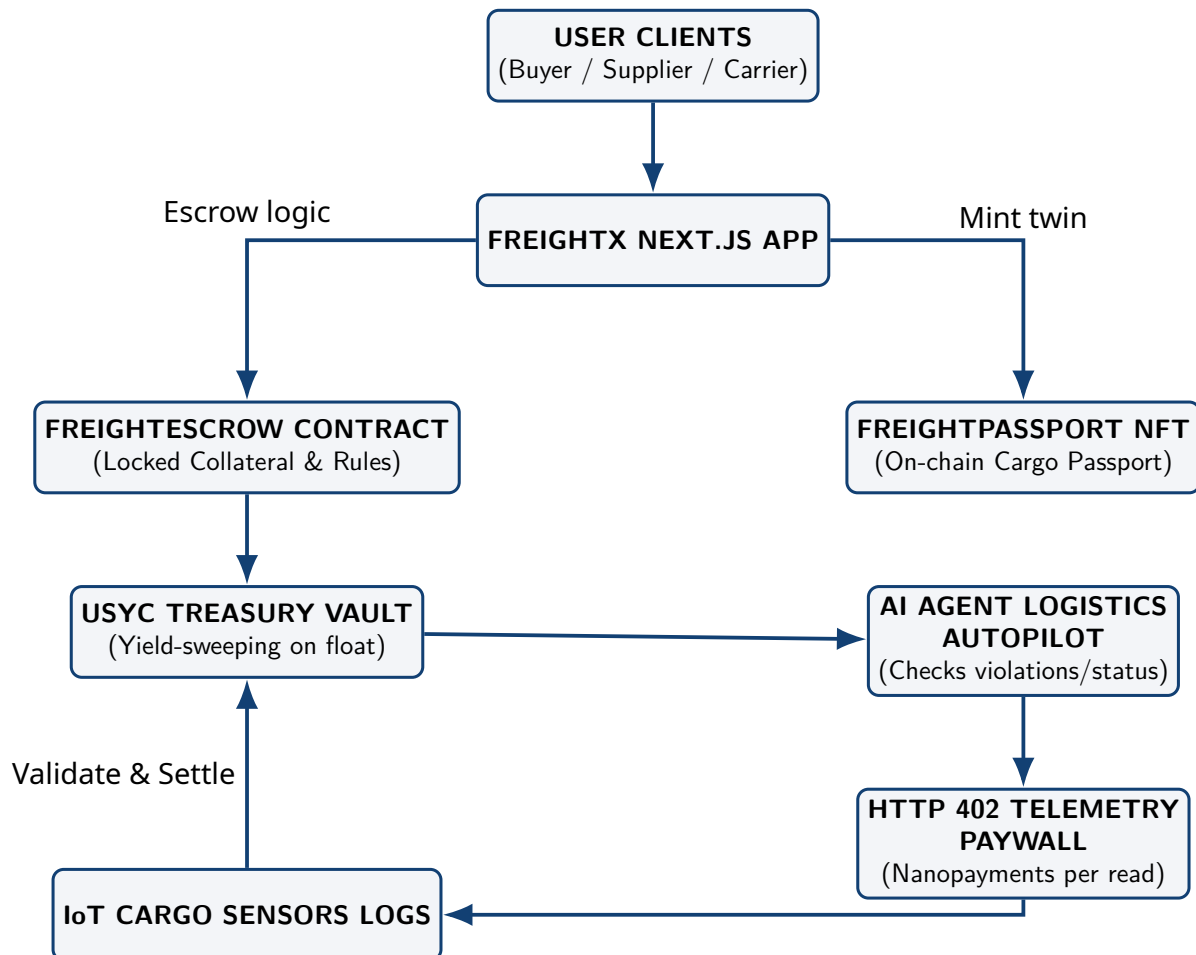


Figure 1: FreightX Protocol Architecture & On-Chain Interaction Model

3.2 Detailed Protocol Workflow

Step 1: Purchase Order Financing

- The Supplier uploads a Purchase Order to the PO Finance Marketplace.
- An Investor funds the loan request using USDC. The loan state changes to “Funded” and the USDC capital is immediately disbursed to the Supplier.

Step 2: Cargo Escrow Booking & Digital Twin Minting

- The Buyer books a cargo shipment, inputs details (Shipper, Consignee, Cargo Value, Shipping Fee, temperature parameters).
- The Buyer checks “Link to PO Loan”. When the transaction is sent to the `FreightEscrow` smart contract, the contract locks the cargo value + shipping fees in USDC.
- Upon successful locking, the linked PO loan is flagged as “Repaid” automatically through on-chain state updates.
- Concurrently, a Bill of Lading NFT is minted via the `FreightPassport` ERC-721 contract to represent the digital cargo twin.

Step 3: Collateral Yield Sweeping

- While the cargo is in transit, the locked escrow USDC is swept into the USYC yield-bearing vault contract.
- The locked funds dynamically accumulate simulated interest based on an APY percentage, turning idle shipping time into passive revenue.

Step 4: Autonomous Telemetry & Paywall

- The cargo container streams GPS location, time, and temperature logs.
- To prevent denial-of-service and charge for telemetry usage, the logs are gated behind a server-side HTTP 402 Paywall.
- The AI Agent Logistics Coordinator queries this telemetry by executing micro-USDC nanopayments. The unlocked telemetry feeds into the AI Agent’s decision tree.

Step 5: Milestone Triggers & Penalties

- When the container passes the Singapore Hub, the AI Agent triggers the “Singapore Milestone” on-chain. The escrow contract instantly releases 30% of the cargo value to the Supplier.
- If the container’s temperature exceeds the threshold (e.g. 8°C), the AI Agent records a thermal breach.
- When the container arrives at the destination and undergoes customs clearance, if it exceeds the free storage time, the demurrage timer accrues a dollar penalty per hour.

Step 6: Final Settlement

- Once customs clearance is complete and the container is picked up, the final settlement is triggered.
- The Carrier receives the shipping fees.
- The Supplier receives the remaining 70% of the cargo value, minus any thermal or demurrage penalties.
- The accumulated USYC interest is refunded directly to the Buyer's wallet.

4 Detailed Technical Stack & Smart Contracts

FreightX is engineered with a robust production-grade stack:

■ Smart Contracts (EVM / Solidity):

- Deployed on Arc Testnet at custom RPC endpoint.
- Uses OpenZeppelin `AccessControl` to enforce operational separation of duties (`Minter` Roles for document managers, `Operator` Roles for transporters).
- Contracts handle multi-party distributions, USYC yield sweeping, and demurrage waterfalls.

■ Frontend & Framework:

- Built on Next.js 16.2.7 and React 19.2.7.
- TypeScript for compilation safety.
- Tailored glassmorphism UI styles utilizing HSL variables and Vanilla CSS.
- Responsive layouts optimized for desktops and logistics tablets.

■ Blockchain & Smart Wallet Client Interfaces:

- Viem (v2.50.4) for low-level JSON-RPC transport handling.
- Wagmi (v2.19.5) and RainbowKit (v2.2.11) for Web3 wallet connectivity.
- Support for smart account clients using Circle Web3 SDKs.

■ Middleware and Storage:

- Next.js Edge Middleware for routing logic.
- Next.js server-side API proxy routes (e.g., `/api/ipfs/upload`) to bypass browser-level CORS limitations during IPFS uploads.
- Server-Sent Events (SSE) via `/api/iot/stream` to deliver real-time IoT telemetry updates to the dashboard without pulling requests repeatedly.
- Supabase integration for indexing on-chain state with fallback in-memory DB store for offline and sandbox testing.

5 Circle Developer Products Integration Details

FreightX achieves its capabilities by integrating six core developer tools:

1. USDC (Core Settlement Rail):

USDC acts as the stable asset throughout the protocol. All escrow contracts, repayments, factoring deals, and crew payouts settle in USDC. On Arc Network, USDC is also used as the gas token, removing the need for a separate volatile native gas asset.

2. Circle Developer-Controlled Wallets:

Used to create and manage system oracle wallets. A secure developer-controlled wallet initiates transactions for the telemetry coordinator and AI Agent without manual user approval.

3. Circle Gateway:

Orchestrates multi-party payouts. When the final settlement is triggered, Gateway payouts distribute funds concurrently to the Supplier, Carrier, and subcontractors.

4. USYC Treasury Sweep:

Escrow collateral locked in `FreightEscrow` is swept to earn interest. This provides buyers with a cash management tool to offset high logistics fees.

5. StableFX:

Provides currency exchange services when generating international invoices, routing payouts between USDC and EURC based on real-time trade FX rates.

6. Nanopayments:

Implements micropayment paywalls for API requests. High-frequency IoT log queries require sub-cent transactions, handled through Circle's payment routing.

6 Exhaustive Circle Product Feedback & Recommendations

This feedback covers our development experience on Arc Testnet:

A. Why we chose these products for our use case

Supply chain logistics involve multiple non-crypto-native companies (carriers, customs brokers, warehouse staff). Volatile gas tokens (like ETH) create onboarding friction. Arc's capability to settle transactions in USDC with predictable, dollar-denominated gas fees was a primary reason for this choice. Additionally, integrating USYC yield-bearing sweeping provided a clear financial incentive for buyers to lock funds inside our escrow contract.

B. What worked well during development

- The JSON-RPC endpoint on Arc Testnet had sub-second finality.
- Integration with Wagmi & Viem was seamless, behaving like standard EVM networks, allowing us to reuse open-source libraries.
- Using USDC directly for gas was simple and worked out-of-the-box.

C. What could be improved & Recommendations

1. IPFS Browser CORS Policy Restrictions

- **Issue:** When saving W3C Verifiable Credentials and Cargo Twin NFT metadata directly from the user's browser to IPFS/Pinata, browser clients regularly revert due to strict CORS (Cross-Origin Resource Sharing) restrictions.
- **Solution implemented:** We built a Next.js server-side API proxy route (`/api/ipfs/upload`) to upload files from the server.
- **Recommendation:** Provide pre-configured, CORS-compliant upload gateway APIs directly in the Circle App-Kit/Bridge-Kit SDKs to save developers from building custom backend proxy upload scripts.

2. AccessControl Revert Logging on Arc Explorer

- **Issue:** When implementing smart contracts with strict OpenZeppelin roles (such as `MINTER_ROLE` or `OPERATOR_ROLE` for minting digital twins), if a transaction is initiated by an unauthorized wallet, the transaction reverts silently with opaque execution details on the explorer.
- **Solution implemented:** We wrote custom Node.js validation scripts to check on-chain role ownership before sending transactions.
- **Recommendation:** Include a contract debugger or a clear transaction trace panel in the Circle Developer Console for the Arc Testnet, making it easier to identify the exact role missing when a transaction reverts.

7 Setup, Deployment & Verification Guide

To run and verify the FreightX application locally:

1. Prerequisites

- Node.js version $\geq$ 18.0.
- NPM package manager.
- A Web3 browser wallet (MetaMask) or biometric hardware for Passkeys.

2. Environment Variables

Create a `.env` file at the root of the project with the following structure:

Listing 1: `.env` configuration file

```
PRIVATE_KEY=your_private_key_here
ARC_TESTNET_RPC_URL=https://rpc.testnet.arc.network
NEXT_PUBLIC_CIRCLE_CLIENT_KEY=your_circle_client_key_here
NEXT_PUBLIC_CIRCLE_CLIENT_URL=https://api.circle.com/v1/w3s
CIRCLE_API_KEY=your_circle_api_key_here
GATEWAY_SELLER_ADDRESS=your_seller_wallet_address_here
NEXT_PUBLIC_GATEWAY_SELLER_ADDRESS=your_seller_wallet_address_here
```

Note: You can leave Supabase variables blank to use the built-in in-memory database fallback.

3. Install & Start Server

Execute the following commands in your terminal:

```
npm install
npm run dev
```

Open <http://localhost:3000> in your web browser.

4. Run Automated Seeding

To populate the database tables with mock shipments and PO loans:

```
node scripts/seedSupabase.js
```

5. Verify Smart Contract State

Verify roles and owners using our included diagnostic scripts:

```
node scripts/check-owner.js
node scripts/check-passport-roles.js
```